Tranky

Guía técnica del backend: cómo funciona por dentro, paso a paso

Versión 0.1 · 25 de octubre de 2026 · Basada en FSD-002 (Portal de Residentes de Edificios)

> Esta guía explica el backend de Tranky **desde que llega una petición a** `app.js` hasta que se guarda en la base de datos y vuelve la respuesta.
>
> Describe **qué hace cada componente**, **cómo se comunican** entre sí y recorre **los flujos principales del sistema**: login, consulta de deudas, pago, confirmación por la pasarela, link de WhatsApp, emisión de cuotas, importación del padrón, mora y notificaciones.
>
> Cada sección técnica trae un recuadro **«En palabras simples»** para quien no programa.

Contenido

1. La idea general en un minuto
2. El mapa del sistema por capas
3. El arranque: `server.js`
4. La puerta de entrada: `app.js`
5. La vida de una petición, paso a paso
6. Componentes transversales (config, db, middleware, shared)
7. Los módulos de negocio, uno por uno
8. Flujo: inicio de sesión (contraseña y OTP)
9. Flujo: consultar unidades, propiedades y deudas
10. Flujo: pagar cuotas (checkout)
11. Flujo: confirmación por webhook
12. Flujo: rechazos, expiración y CIP
13. Flujo: pago desde link de WhatsApp
14. Flujo: emisión de cuotas y avisos
15. Flujo: importación del padrón (.xlsx)
16. Flujo: mora diaria y jobs programados
17. Flujo: notificaciones (outbox)
18. Flujo: manejo de errores
19. La base de datos y sus protecciones
20. Estados de cuotas y pagos
21. Cómo extender el sistema
22. Glosario

1. La idea general en un minuto

El backend es un **servidor web** (una API) que recibe pedidos de dos aplicaciones:

- **Tranky (app del residente):** el propietario o inquilino consulta sus deudas y paga.
- **Portal de la administradora:** registra edificios y residentes, emite las cuotas y mira la cobranza.

Para cada pedido, el backend verifica **quién es** (autenticación), **qué puede hacer** (permisos), aplica las **reglas del negocio** y lee o escribe en la **base de datos PostgreSQL**. Para cobrar habla con una **pasarela de pagos**; para avisar al residente usa **WhatsApp, SMS o email**.

> **EN PALABRAS SIMPLES**
> Piensa en el backend como la oficina de la administración del edificio: tiene una recepción (`app.js`) que atiende a todos, ventanillas especializadas (los módulos: pagos, residentes, administradora), un archivo central (la base de datos) y un cajero que confirma cada pago con el banco (la pasarela). Nada se da por pagado hasta que el banco lo confirma.

Tecnologías usadas

Pieza	Tecnología	Para qué
Servidor	Node.js 22 + Express 5	Recibir peticiones HTTP y responder JSON o PDF
Base de datos	PostgreSQL 16 (Docker)	Guardar todo, con transacciones seguras y campos JSON flexibles
Acceso a datos	`pg` (SQL directo)	Consultas explícitas, bloqueos y transacciones controladas a mano
Validación	Zod 4	Revisar que lo que llega tenga el formato correcto (mensajes en español)
Sesiones	JWT + bcrypt	Tokens de acceso y contraseñas cifradas
Tareas programadas	node-cron	Mora diaria, expiración de pagos y envío de notificaciones
Documentos	pdfkit · exceljs	Avisos y comprobantes en PDF · leer y crear Excel
Registro (logs)	pino	Bitácora en JSON con un identificador por petición

2. El mapa del sistema por capas

El código se organiza en capas. Cada capa solo habla con la de abajo, y eso hace que sea fácil saber dónde buscar cada cosa.

Clientes	App del residente (PWA), portal de la administradora y la pasarela de pagos (envía webhooks)
▼ HTTP / JSON	
Entrada · `app.js`	Recibe todo, aplica la seguridad común (registro, cabeceras seguras, CORS, límite de peticiones) y deriva a cada grupo de rutas
▼	
Rutas · `*.routes.js`	Una por módulo. Verifican el token (`authenticate`), validan los datos con Zod y llaman al servicio
▼	
Servicios · `*.service.js`	Las reglas del negocio: permisos por rol, cálculo de montos, estados de pagos, PDFs, notificaciones. Abren transacciones de base de datos
▼ SQL	
Base de datos · PostgreSQL	Tablas, restricciones y **triggers** que impiden borrar o alterar pagos (última línea de defensa)

A los costados de estas capas hay **componentes transversales** que usan todos: la configuración (`config/`), la conexión a la base (`db/`), los middlewares (`middleware/`) y las utilidades (`shared/`). Además, los **jobs** (`jobs/`) ejecutan tareas programadas usando los mismos servicios, sin pasar por HTTP.

Estructura de carpetas

Carpeta	Contenido
`src/server.js`	Punto de arranque del proceso
`src/app.js`	Arma la aplicación Express: middlewares y rutas
`src/config/`	`env.js` (lee el `.env`), `rubros.js` (vocabulario inmobiliario), `organizationUnitConfig.js` (reglas por edificio)
`src/db/`	Conexión (`pool.js`) y scripts: `create`, `migrate`, `reset`, `seed`
`src/middleware/`	`auth.js` (tokens), `requestContext.js` (registro), `rateLimits.js` (límites)
`src/shared/`	Errores, montos, seguridad, auditoría, validación, logger
`src/modules/`	Un módulo por área: `auth`, `payer`, `payments`, `webhooks`, `gateways`, `payLinks`, `debts`, `notifications`, `documents`, `tenant`, `dev`
`src/jobs/`	Programación de tareas automáticas
`db/migrations/`	Definición de las tablas y triggers (SQL versionado)
`tests/`	34 pruebas automáticas de punta a punta

3. El arranque: `server.js`

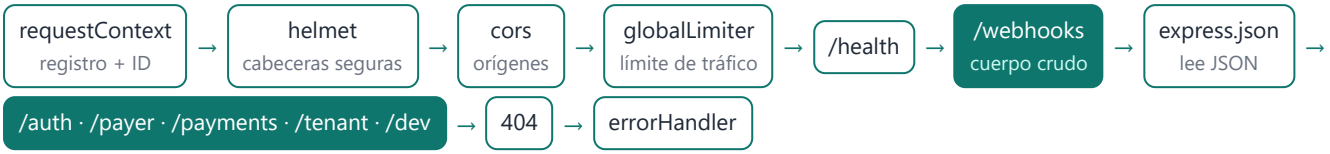
Cuando ejecutas `npm run dev`, Node corre `src/server.js`. Esto es lo que pasa, en orden:

1	`config/env.js`	Lee el archivo `.env` y arma el objeto `env` con toda la configuración (puerto, base de datos, secretos, tiempos). Si falta una variable obligatoria (`DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `OTP_PEPPER`) **detiene el arranque** con un mensaje claro.
2	`db/pool.js`	Crea el **pool de conexiones** a PostgreSQL (hasta 20 conexiones reutilizables). Fija la zona horaria de cada conexión en `America/Lima`, para que «hoy» y los vencimientos se calculen en hora peruana.
3	`server.js`	Hace una consulta de prueba (`SELECT 1`). Si la base no responde, el servidor no arranca.
4	`jobs/index.js`	`iniciarJobs()` programa las tres tareas automáticas: mora diaria (00:01), expiración de pagos (cada minuto) y envío de notificaciones (cada 15 segundos).
5	`app.js`	`createApp()` construye la aplicación Express (capítulo 4) y `server.js` la pone a escuchar en el puerto `PORT` (3001).
6	`server.js`	Queda atento a las señales de apagado (`Ctrl + C`): detiene los jobs, deja de aceptar peticiones y cierra las conexiones a la base de forma ordenada.

> **EN PALABRAS SIMPLES**
> Antes de abrir la oficina, el sistema revisa que tenga sus llaves (configuración), que el archivo central esté accesible (base de datos) y programa sus tareas de rutina. Recién entonces abre la puerta al público.

4. La puerta de entrada: `app.js`

`app.js` define una **cadena de middlewares**: funciones por las que pasa cada petición, en orden, antes de llegar a su destino. **El orden importa.**



Paso	Qué hace y por qué está en ese lugar
`trust proxy`, sin `x-powered-by`	Toma la IP real del cliente si hay un proxy delante y oculta qué tecnología usa el servidor.
`requestContext`	**Primero de todos**, para que cada petición, incluso las que fallen después, quede registrada. Asigna un **correlation-id** (identificador único) que se devuelve en la cabecera `X-Correlation-Id` y acompaña cada línea de log. Al terminar escribe: método, ruta, estado HTTP y milisegundos.
`helmet`	Agrega cabeceras de seguridad estándar a las respuestas.
`cors`	Permite que la app web, alojada en otro dominio, llame a la API, y le deja leer el `X-Correlation-Id`.
`globalLimiter`	Máximo 600 peticiones cada 15 minutos por IP. Frena abusos antes de gastar recursos.
`GET /health`	Responde «ok» sin tocar la base. Sirve para monitoreo.
`/api/v1/webhooks`	**Antes del lector de JSON a propósito.** La firma de seguridad (HMAC) de la pasarela se calcula sobre los bytes exactos recibidos; si Express los convirtiera a JSON antes, la firma ya no se podría verificar.
`express.json`	Convierte el cuerpo JSON de las demás peticiones en un objeto (máximo 1 MB).
Rutas de negocio	`/auth` (login), `/payer` (residente), `/payments` (pagos), `/tenant` (administradora). `/dev` solo se monta si `ENABLE_DEV_ROUTES=true` y fuera de producción.
404	Si ninguna ruta coincidió, responde «Ruta no encontrada» con el formato de error estándar.
`errorHandler`	**Último de todos.** Atrapa cualquier error lanzado en el camino y lo convierte en una respuesta clara (capítulo 18). Gracias a Express 5, los errores de funciones asíncronas llegan aquí solos.

> **EN PALABRAS SIMPLES**
> Es como la recepción de un edificio de oficinas: anotan tu visita con un número de ticket, revisan tu bolso, verifican que no estés entrando cien veces por minuto y te mandan a la ventanilla correcta. Si algo sale mal en cualquier ventanilla, siempre hay un encargado al final que te explica qué pasó.

5. La vida de una petición, paso a paso

Ejemplo real: el residente abre «Mis unidades» y la app llama a `GET /api/v1/payer/units` con su token.

1	`requestContext.js`	Asigna el correlation-id y prepara el registro de la petición.
2	`helmet · cors · limiter`	Cabeceras de seguridad, permiso de origen y control de cantidad de peticiones.
3	`express.json`	No hay cuerpo (es un GET), así que pasa directo.
4	`app.js`	La ruta empieza con `/api/v1/payer`, así que deriva al router `payer.routes.js`.
5	`middleware/auth.js`	`authenticate(TOKEN.PAYER)` lee `Authorization: Bearer ...`, verifica la firma y la vigencia del JWT y que sea de tipo `PAYER`. Guarda en `req.auth` quién es (`sub` = id del pagador). Si falla: 401 o 403.
6	`payer.routes.js`	La ruta `GET /units` llama a `listarUnidades(req.auth.sub)`.
7	`payer.service.js`	Pide a `access.service.js` las unidades del pagador.
8	`access.service.js`	`unidadesDelPagador()` ejecuta un SQL que une relaciones activas, unidades, edificios y administradoras activas. Para cada unidad resuelve la configuración del edificio (`resolverConfig`) y los permisos efectivos de sus roles (`permisosEfectivos`).
9	`payer.service.js`	`agruparPorEdificio()` ordena el resultado en edificio → tipo (Departamentos, Depósitos, Cocheras) → unidades.
10	`presenters.js · rubros.js`	`resumenUnidad()` traduce los datos técnicos a texto para mostrar, por ejemplo «Cochera techada · N.° 07 · Nivel S2», usando el catálogo del rubro.
11	`Express`	Responde 200 con el JSON. `requestContext` escribe la línea de log con el estado y los milisegundos.

EN PALABRAS SIMPLES

Toda consulta del residente pasa por un mismo «filtro de acceso» (`access.service.js`) que solo muestra las unidades donde tiene una relación vigente. Si un inquilino terminó su contrato, deja de ver esa unidad automáticamente.

6. Componentes transversales

Son piezas que no pertenecen a un módulo en particular: las usan todos.

6.1 Configuración (`src/config`)

env.js

`src/config/env.js`

Único lugar que lee variables de entorno. Convierte números y booleanos, aplica valores por defecto no sensibles y exige las credenciales. Ningún otro archivo lee `process.env` directamente (salvo los scripts de base y el seed).

rubros.js

`src/config/rubros.js`

El **catálogo del rubro inmobiliario**: tipos de unidad (DEPARTAMENTO, DEPOSITO, COCHERA) con sus etiquetas, los atributos permitidos en `metadata_json` (piso, nivel, techada, alícuota…) y cómo describirlos, más los tipos de cuota (ORDINARIA, EXTRAORDINARIA). Funciones: `validarMetadata`, `presentarUnidad`, `grupoTipoUnidad`, `validarTipoCuota`.

Así la base de datos queda genérica y el mismo núcleo puede servir a otros rubros (colegios, clubes) agregando otro catálogo.

organizationUnitConfig.js

`src/config/organizationUnitConfig.js`

Las **reglas configurables por edificio**: regla de mora y permisos por rol (qué tipos de cuota ve y paga cada rol, y si ve el historial de toda la unidad o solo el suyo). `resolverConfig` mezcla lo configurado con los valores por defecto; `permisosEfectivos` une los permisos de todos los roles de una persona en una unidad; `puedeVerCuota` y `puedePagarCuota` deciden cada caso.

6.2 Base de datos (`src/db`)

Archivo	Qué hace
`pool.js`	Pool de conexiones y la función `withTransaction(fn)`: abre `BEGIN`, ejecuta `fn` con una conexión dedicada y hace `COMMIT`, o `ROLLBACK` si algo falla. Todas las operaciones que deben ser «todo o nada» la usan.
`create.js`	`npm run db:create`: crea o sincroniza el rol y las bases dentro del contenedor Docker usando `db/bootstrap.sql` y los valores del `.env`, enviados por la entrada estándar para no exponer la contraseña.
`migrate.js`	Aplica en orden los archivos de `db/migrations` que falten, cada uno en su propia transacción, y los anota en `schema_migrations`. Usa un candado para que dos procesos no migren a la vez.
`reset.js`	Borra y recrea el esquema completo (solo desarrollo).
`seed.js`	Carga datos de prueba: 2 administradoras, 2 edificios, 5 unidades, 4 residentes y 9 cuotas, y luego calcula la mora.

6.3 Middlewares (`src/middleware`)

Archivo	Qué hace
`auth.js`	Emite y verifica los tres tipos de token JWT: `PAYER` (residente, 8 h), `TENANT_ADMIN` (administradora, 8 h, incluye su `tenant_id`) y `PAY_LINK` (15 min, limitado a una cuota). `authenticate(...tipos)` rechaza con 401 un token ausente, inválido o vencido, y con 403 un tipo no permitido en esa ruta.
`requestContext.js`	Correlation-id, logger por petición (con el `Idempotency-Key` si viene) y línea de log al terminar.
`rateLimits.js`	Límites: global (600/15 min), login (20/15 min), OTP (5/15 min), checkout (20/min) y links de pago (60/15 min). Se desactivan en pruebas.

6.4 Utilidades (`src/shared`)

Archivo	Qué hace
`errors.js`	`AppError` (error con estado HTTP, código y mensaje), `notFound` y el `errorHandler` final.
`money.js`	Montos en **céntimos enteros** (`toCents` , `fromCents` , `roundCents`) para que 0,10 + 0,20 dé exactamente 0,30. `formatMoney` muestra «S/ 1,234.50».
`security.js`	Hash SHA-256, firma HMAC, comparación en tiempo constante, tokens y dígitos aleatorios criptográficos, y el código de operación (`TRK-AAAAMMDD-XXXXXXXXXX`).
`audit.js`	Escribe en `audit_logs` quién hizo qué (login, checkout, cambios del padrón), con su IP y correlation-id. Puede escribir dentro de la misma transacción de la operación.
`validation.js`	Activa los mensajes de Zod en español y define validadores comunes: UUID, periodo `YYYY-MM` , fecha, documento (DNI 8 dígitos, CE, RUC 11), celular peruano, email y paginación.
`logger.js`	Logger `pino` en JSON que **oculta** contraseñas, OTP, tokens de tarjeta y la cabecera `Authorization` .

7. Los módulos de negocio, uno por uno

Cada módulo vive en `src/modules/<nombre>`. Las **rutas** traducen HTTP; los **servicios** contienen la lógica.

auth · inicio de sesión — `auth.routes.js · auth.service.js`

`loginPagador` (documento + contraseña u OTP), `solicitarOtp` (genera y envía el código), `loginAdministrador` y `cambiarPassword`. Lleva el conteo de intentos fallidos y bloquea temporalmente la cuenta.

Interactúa con: `middleware/auth.js` (emite tokens), `notifications/providers.js` (envía el OTP), `shared/audit.js`.

payer · portal del residente — `payer.routes.js · access.service.js · payer.service.js · presenters.js · paymentMethods.service.js`

- `access.service.js`: **el filtro de acceso**. Devuelve solo las unidades con relación activa y sus permisos.
- `payer.service.js`: unidades, «Mis propiedades» con saldos, dashboard, cuotas por unidad, historial paginado, datos para los PDFs y reporte de dato incorrecto.
- `presenters.js`: arma los objetos de respuesta (cuota, unidad) con etiquetas legibles y el indicador `puede_pagar`.
- `paymentMethods.service.js`: billetera de tarjetas (listar, agregar, cambiar la predeterminada, débito automático y baja lógica).

Interactúa con: `config/organizationUnitConfig.js`, `config/rubros.js`, `documents/pdf.service.js`, `payLinks`, `notifications`.

payments · motor de pagos — `payments.routes.js · checkout.service.js · fees.js · paymentState.service.js · expiration.job.js`

- `checkout.service.js`: `checkout()` (crear el intento de pago) y `estadoPago()` (consulta para el polling).
- `fees.js`: `calcularMontos()`, que calcula la comisión de la recaudadora, el IGV de la comisión, la comisión de la pasarela, el total cobrado y el neto a liquidar.
- `paymentState.service.js`: las transiciones de estado: `marcarExitoso`, `marcarFallido` y `liberarCuotas`.
- `expiration.job.js`: `expirarIntentos()` libera los intentos vencidos.

Interactúa con: `gateways` (cobrar), `payer/access.service.js` (permisos), `debts/mora.service.js` (recalcular al liberar), `notifications` (recibo o aviso de rechazo).

gateways · pasarelas de pago — `PaymentGateway.js · MockGateway.js · registry.js`

Patrón **Strategy**: `PaymentGateway` define el contrato (`createCharge`, `getCharge`, `refund`, `estimateFee`, `verifyWebhook`, `parseWebhook`). `MockGateway` simula una pasarela real en memoria. `registry.js` decide qué adaptador usar según el método de pago (`PASARELA_YAPE`, `PASARELA_CARD`, `PASARELA_CIP` en el `.env`).

webhooks · confirmaciones de la pasarela — `webhooks.routes.js · webhook.service.js`

`procesarWebhook()` valida la firma HMAC, descarta eventos repetidos, ubica la transacción y aplica el resultado con `paymentState.service.js`. Es la **única puerta** por la que una cuota pasa a «Pagada».

payLinks · links de pago de WhatsApp `payLinks.service.js`

`crearPayLink()` genera un token aleatorio y guarda **solo su hash**. `resolverPayLink()` lo valida y entrega un token `PAY_LINK` de 15 minutos para pagar esa única cuota.

debts · mora `mora.service.js`

`calcularMoraCents()` aplica la regla del edificio; `recalcularMora()` pasa cuotas a VENCIDO y actualiza la mora. Lo usan el job diario, la liberación de cuotas, la emisión y el seed.

notifications · avisos y recibos `notification.service.js · dispatcher.js · providers.js`

`encolarParaPagador` y `encolarParaAdministradores` guardan mensajes en la tabla `notifications`. `procesarNotificaciones` (job) los arma con su plantilla y los entrega al proveedor de cada canal. `providers.js` hoy usa un proveedor de consola que escribe en el log.

documents · PDFs `pdf.service.js`

`renderAvisoCobranza()` y `renderComprobante()` generan los PDF con `pdfkit` y los envían directamente en la respuesta.

tenant · portal de la administradora `tenant.routes.js · padron.service.js · import.service.js · emision.service.js · monitor.service.js`

- `padron.service.js`: edificios, unidades, residentes y sus vínculos. Toda consulta filtra por el `tenant_id` del token.
- `import.service.js`: lectura y validación del Excel del padrón y su aplicación «todo o nada».
- `emision.service.js`: `emitirCuotas()`, la emisión masiva idempotente.
- `monitor.service.js`: KPIs, listados de cuotas, transacciones, reportes y liquidaciones.

dev · herramientas de desarrollo `dev.routes.js`

Simula el webhook de la pasarela, genera links de pago de prueba y ejecuta jobs al instante. No existe en producción.

Mapa de interacciones entre módulos

Módulo	Usa a…
`payments`	`gateways`, `payer/access`, `debts`, `notifications`, `shared/audit`
`webhooks`	`gateways`, `payments/paymentState`
`payer`	`config`, `documents`, `payLinks`, `notifications`, `auth` (cambio de contraseña)
`payLinks`	`payer/access`, `payer/presenters`, `middleware/auth`
`tenant`	`config`, `debts`, `notifications`
`notifications`	`payLinks` (crea el link al enviar el aviso)
`jobs`	`debts`, `payments/expiration`, `notifications/dispatcher`

8. Flujo: inicio de sesión

8.1 Con documento y contraseña — `POST /api/v1/auth/login`

1	`rateLimits.js`	`loginLimiter`: máximo 20 intentos cada 15 minutos por IP.
2	`auth.routes.js`	Zod valida el tipo y número de documento y exige **contraseña u OTP, no ambos**.
3	`auth.service.js`	`loginPagador()` busca al pagador por documento. Si está bloqueado (`bloqueado_hasta` futuro) responde 429.
4	`bcrypt`	Compara la contraseña con el hash guardado. **Si el documento no existe, compara igual contra un hash aleatorio**: así la respuesta tarda lo mismo y no revela si el DNI está registrado.
!	`auth.service.js`	Si falla: suma un intento (al quinto bloquea 15 minutos), audita «LOGIN_FALLIDO» y responde 401 con un mensaje genérico.
5	`middleware/auth.js`	Si es correcta: reinicia los intentos, audita «LOGIN_OK» y emite un token `PAYER` de 8 horas con `signPayerToken()`.

8.2 Con código OTP por WhatsApp o SMS

1	`POST /auth/otp`	`solicitarOtp()`: si el documento existe, tiene teléfono y no pidió un código hace menos de 60 s, genera 6 dígitos aleatorios. **Siempre** responde el mismo mensaje, exista o no.
2	`otp_challenges`	Invalida los códigos anteriores y guarda el nuevo como **hash HMAC con un secreto** (`OTP_PEPPER`), con vencimiento de 5 minutos.
3	`providers.js`	Envía el código por el canal elegido. En desarrollo, además lo devuelve en `dev_otp`.
4	`POST /auth/login`	Con `otp`: dentro de una transacción bloquea el código vigente, lo compara en tiempo constante y lo marca usado. Al tercer error lo invalida.
5	`middleware/auth.js`	Emite el token `PAYER`, igual que con contraseña.

> **EN PALABRAS SIMPLES**
> El sistema nunca dice «ese DNI no existe»: siempre responde lo mismo, para que nadie pueda averiguar quién vive en un edificio probando documentos. Las contraseñas y los códigos se guardan cifrados de forma que ni siquiera el equipo técnico puede leerlos.

9. Flujo: consultar unidades, propiedades y deudas

Las cuatro pantallas de consulta del residente comparten la misma base: `access.service.js` obtiene las unidades visibles con sus permisos y después cada servicio filtra y arma su respuesta.

Endpoint	Función	Qué hace después del filtro de acceso
`GET /payer/units`	`listarUnidades`	Agrupa por edificio y tipo. Indica si hace falta mostrar el selector (más de una unidad).
`GET /payer/properties`	`misPropiedades`	Busca las cuotas abiertas visibles, calcula el saldo pendiente, vencido y en proceso por unidad, el subtotal por edificio y el total general (por moneda), y el último pago de cada unidad respetando el alcance del historial.
`GET /payer/debts`	`dashboardDeuda`	Lo mismo para el dashboard: deuda total, vencida, en proceso y próximo vencimiento. Si no hay nada pagable responde «Estás al día».
`GET /payer/units/:id/debts`	`cuotasDeUnidad`	Verifica que la unidad sea del residente (si no, 404) y lista sus cuotas por vencimiento ascendente.
`GET /payer/units/:id/transactions`	`historialPagos`	Pagos confirmados o extornados, paginados, con el total histórico y sus extornos vinculados.

Cómo se decide qué cuotas ve cada residente

1. `access.service.js` — Obtiene los roles activos del residente en cada unidad, por ejemplo `["INQUILINO"]`.
2. `organizationUnitConfig` — `resolverConfig(edificio.config_json)` toma la configuración del edificio o los valores por defecto. `permisosEfectivos()` calcula los conjuntos `ver` y `pagar` y el alcance del historial.
3. `payer.service.js` — `cuotasVisibles()` trae las cuotas de esas unidades y descarta las que no pasan `puedeVerCuota()`. También descarta las dirigidas a otra persona (`debts.payer_id`).
4. `presenters.js` — `presentarCuota()` marca `puede_pagar`: verdadero solo si la cuota está PENDIENTE o VENCIDA y el rol puede pagar ese tipo de cuota.

Historial: «toda la unidad» o «solo mis pagos»

`alcanceHistorial()` separa las unidades en dos grupos según el permiso `historial` del rol. El SQL del historial trae los pagos de toda la unidad para el primer grupo y **solo los pagos hechos por el propio residente** para el segundo. Por defecto, el propietario ve todo y el inquilino solo lo suyo.

PDFs

`GET /payer/debts/:id/notice.pdf` y `GET /payer/transactions/:id/receipt.pdf` verifican la visibilidad con el mismo filtro y luego `pdf.service.js` dibuja el documento y lo envía en la respuesta. El comprobante solo existe para pagos confirmados.

10. Flujo: pagar cuotas (checkout)

El corazón del sistema. Endpoint: `POST /api/v1/payments/checkout`, con la cabecera `Idempotency-Key`. Archivo principal: `payments/checkout.service.js`.

Fase A — Antes de tocar la base de datos

1	`rateLimits · auth`	`checkoutLimiter` (20 por minuto) y `authenticate(PAYER, PAY_LINK)`.
2	`checkout()`	Exige un `Idempotency-Key` de 16 a 80 caracteres. Valida el cuerpo con `checkoutSchema`: de 1 a 24 cuotas sin repetir, método YAPE/CARD/CIP, celular y OTP de 6 dígitos para Yape, y para tarjeta **o** una tarjeta guardada **o** un token nuevo. Rechaza cualquier valor que parezca un número de tarjeta. **Cualquier monto que envíe la app se ignora.**
3	`validarPayLink()`	Si el token es de link de WhatsApp: solo puede pagar su cuota, no puede usar ni guardar tarjetas y el link debe seguir vigente.
4	`hashSolicitud()`	Calcula una huella (SHA-256) de la solicitud para detectar si alguien reusa la misma llave con otro contenido.
5	`replay()`	**¿Esa llave ya se usó?** Si es del mismo residente y la misma solicitud, devuelve el mismo pago sin cobrar de nuevo. Si es de otro residente: 409. Si cambió el contenido: 422.
6	`registry.js`	`gatewayParaMetodo()` elige la pasarela según el método de pago.

Fase B — Transacción de base de datos (todo o nada)

7	`SELECT … FOR UPDATE`	**Bloquea las cuotas** en orden por id, así dos pagos simultáneos no se cruzan ni se traban entre sí. Si otro residente está pagando la misma cuota, esta petición espera a que termine.
8	`access.service.js`	Recalcula las unidades y permisos del residente para esas cuotas.
!	`validaciones`	Cuota invisible: 404. `EN_PROCESO`: 409 «Esta cuota tiene un pago en proceso». Pagada o anulada: 409. Rol sin permiso: 403. Cuotas de distinto edificio o moneda: 422.
9	`payment_methods_vault`	Si es tarjeta guardada, la busca (debe ser del residente y estar activa). Si es tarjeta nueva con «guardar», la registra; queda predeterminada si es la primera.
10	`fees.js`	`calcularMontos()` con los montos **de la base de datos** y la configuración de comisión de la administradora (ver recuadro).
11	`INSERT transactions`	Crea la transacción en estado `INICIADO` con todos los montos, la llave, la huella y su vencimiento: 30 minutos para Yape o tarjeta, 24 horas para CIP.
12	`INSERT transaction_debts`	Una fila por cuota con el monto aplicado.
13	`UPDATE debts`	Las cuotas pasan a `EN_PROCESO`. Audita «CHECKOUT_INICIADO». **COMMIT**: se liberan los bloqueos.

Fase C — Hablar con la pasarela (fuera de la transacción)

14	`gateway.createCharge()`	Envía el cobro con el monto total, la moneda, el método, la llave de idempotencia y el id de la transacción como referencia. Se hace **después** del COMMIT para no mantener cuotas bloqueadas mientras se espera a la red.
!	`sin respuesta`	Si la pasarela no contesta, la transacción queda `INICIADO` y el job de expiración la revisará más tarde. Responde 202.
!	`DECLINED`	Rechazo inmediato (por ejemplo, OTP de Yape inválido): `marcarFallido()` deja la transacción FALLIDO y libera las cuotas. Responde **402**; el residente puede reintentar con una llave nueva.
15	`PENDING`	Guarda el id del cargo de la pasarela y los datos extra (código CIP y QR, o el desafío 3DS). Responde **202** con estado `EN_PROCESO` y el resumen de montos.
16	`GET /payments/:id/status`	La app consulta cada pocos segundos (`estadoPago()`) hasta ver PAGADO o FALLIDO.

CÓMO SE CALCULA EL MONTO (FEES.JS)

Deuda = suma de `monto_total` de las cuotas, en céntimos.
Comisión de la recaudadora = monto fijo + porcentaje × deuda (según la administradora).
IGV = comisión × `IGV_TASA` (solo sobre la comisión).
Si **paga el residente**: total = deuda + comisión + IGV + comisión de pasarela, y la junta recibe la deuda completa.
Si **paga la administradora**: total = deuda, y la junta recibe la deuda menos las comisiones.

11. Flujo: confirmación por webhook

La pasarela avisa el resultado llamando a `POST /api/v1/webhooks/gateway/{provider}`. Archivo: `webhooks/webhook.service.js`.

1	`app.js`	La ruta está montada **antes** de `express.json`. `webhooks.routes.js` usa `express.raw` para recibir los bytes exactos.
2	`registry.js`	`getGateway(provider)` obtiene el adaptador. Si el proveedor no existe: 404.
!	`verifyWebhook()`	Recalcula la firma HMAC con el secreto compartido (`WEBHOOK_SECRET_MOCK`) y la compara en tiempo constante. Si no coincide: 401, y queda una alerta `WEBHOOK_FIRMA_INVALIDA` en el log. **Nada cambia.**
3	`parseWebhook()`	Traduce el formato de esa pasarela a un evento estándar: id, tipo, id del cargo, referencia, monto y motivo de fallo.
4	`webhook_events`	Dentro de una transacción inserta el evento con la clave única `(provider, event_id)`. **Si ya existía, responde «duplicado» y no hace nada más**: las pasarelas reenvían eventos y no deben procesarse dos veces.
5	`SELECT … FOR UPDATE`	Busca y bloquea la transacción por id del cargo o por la referencia. Si todavía no existe (la pasarela fue más rápida que el checkout), revierte todo y responde 404 para que la pasarela reintente.
6	`charge.succeeded`	Si ya estaba EXITOSO: «YA_APLICADO». Si estaba cerrada (por ejemplo, expirada): «REQUIERE_CONCILIACION», con alerta para revisión manual. Si el monto no coincide: «MONTO_NO_COINCIDE», con alerta. Si todo está bien: `marcarExitoso()`.
7	`marcarExitoso()`	Transacción → EXITOSO con fecha de pago. Cuotas → PAGADO. El link de pago queda usado (si lo hubo) y se encola el **recibo** (RECIBO_PAGO).
6b	`charge.failed`	`marcarFallido()`: transacción → FALLIDO, cuotas liberadas y aviso de rechazo encolado.
8	`webhook_events`	Anota el resultado y la hora de procesamiento. COMMIT y responde 200 a la pasarela.

EN PALABRAS SIMPLES

El banco o la pasarela envía un «sobre sellado». El sistema revisa el sello; si es falso, lo descarta. Si el mismo sobre llega dos veces, lo procesa una sola vez. Y si el monto del sobre no coincide con lo que se pidió, no marca nada como pagado y avisa a un humano.

Regla de oro: **la única forma de que una cuota quede «Pagada» es un webhook con firma válida**. Ni la respuesta del checkout ni la pantalla de éxito del navegador cuentan.

12. Flujo: rechazos, expiración y código CIP

12.1 Liberar cuotas — `liberarCuotas()`

Cuando un pago falla o vence, cada cuota que estaba `EN_PROCESO` vuelve a **VENCIDO** si su fecha de vencimiento ya pasó, o a **PENDIENTE** si no. Inmediatamente se llama a `recalcularMora()` para que su mora quede al día.

12.2 Expiración automática — `expirarIntentos()` (cada minuto)

1	`expiration.job.js`	Busca transacciones `INICIADO` cuyo `expira_en` ya pasó (hasta 100 por vuelta).
2	`FOR UPDATE SKIP LOCKED`	Bloquea cada una, saltando las que otro proceso esté tocando.
3	`gateway.getCharge()`	Pregunta a la pasarela. Si dice que **sí se cobró**, no la expira: deja una alerta `PAGO_SIN_WEBHOOK` y espera el webhook (regla de oro).
4	`marcarFallido()`	Si no se cobró: FALLIDO con el motivo «Código CIP expirado» o «Tiempo de pago agotado», cuotas liberadas y aviso al residente.

12.3 Pago con CIP o QR

El checkout responde 202 con un **código de 9 dígitos y un QR** válidos 24 horas. Mientras tanto las cuotas quedan «Pago en proceso» para que nadie las pague dos veces. Si el residente paga en su banco, llega el webhook y queda PAGADO. Si no, al vencer el job las libera.

12.4 Tarjeta con 3DS

Si el emisor exige autenticación adicional, la pasarela devuelve `next_action` (tipo 3DS). La app resuelve el desafío con el SDK de la pasarela y el resultado llega, como siempre, por webhook.

13. Flujo: pago desde link de WhatsApp

Permite pagar una cuota en unos 15 segundos sin registrarse (RF-R-14).

1	`dispatcher.js`	Al enviar un aviso de cobranza, `crearPayLink()` genera un token aleatorio de 32 bytes, guarda **solo su hash SHA-256** en `pay_links` (cuota, residente y vencimiento de 72 h) y arma la URL `PAY_LINK_BASE_URL + token`.
2	`WhatsApp`	El residente recibe el mensaje con el link y lo abre en el celular. La app llama a `GET /payer/pay-links/{token}`, que no requiere login.
3	`resolverPayLink()`	Calcula el hash del token recibido y lo busca. Verifica que no esté vencido ni usado, que el residente siga vinculado a la unidad, que su rol pueda pagar esa cuota y que la cuota no esté pagada ni anulada.
4	`signPayLinkToken()`	Devuelve los datos de la cuota y un token `PAY_LINK` de 15 minutos limitado a esa cuota.
5	`checkout()`	La app paga con ese token. El flujo es el del capítulo 10, con las restricciones del link: solo esa cuota y sin tarjetas guardadas.
6	`marcarExitoso()`	Al confirmarse por webhook, el link queda **usado**. Si alguien lo vuelve a abrir recibe 410 «El link expiró o ya fue usado».

EN PALABRAS SIMPLES

El link es como una llave de un solo uso para una sola puerta. Aunque alguien lo reenvíe, solo sirve para pagar esa cuota, vence a los 3 días y deja de funcionar apenas se paga. Como en la base solo se guarda una «huella» del link, ni siquiera quien vea la base de datos puede reconstruirlo.

14. Flujo: emisión de cuotas y avisos

La administradora emite las cuotas del mes con `POST /api/v1/tenant/debts/generate`. Archivo: `tenant/emision.service.js`.

1	`tenant.routes.js`	Token `TENANT_ADMIN`; `emisionSchema` valida edificio, periodo, concepto, tipo de cuota, fechas, modo de cálculo e ítems.
2	`edificioDelTenant()`	Comprueba que el edificio sea de **esa** administradora (si no, 404). `validarTipoCuota()` revisa el catálogo del rubro.
3	`SELECT billing_units`	Unidades activas del edificio, opcionalmente filtradas por tipo o por lista.
4	`itemsParaUnidad()`	**MONTO_FIJO**: cada unidad paga los ítems tal cual. **ALICUOTA**: cada ítem es un total del edificio y se multiplica por la alícuota (%) de la unidad. Si alguna unidad no tiene alícuota: 422 con la lista.
5	`INSERT … ON CONFLICT DO NOTHING`	Crea una cuota por unidad. La clave única (unidad, periodo, concepto) hace que **repetir la emisión no duplique cuotas**.
6	`recalcularMora()`	Por si alguna cuota se emitió con una fecha de vencimiento ya pasada.
7	`encolarParaPagador()`	Para cada cuota nueva y cada residente vinculado **que pueda pagarla según su rol**, encola un AVISO_COBRANZA por cada canal configurado.
8	`audit.js`	Audita «CUOTAS_EMITIDAS» y responde cuántas se crearon, cuántas ya existían y cuántos avisos se encolaron.

Todo ocurre en una sola transacción: si algo falla, no queda ninguna cuota a medias.

15. Flujo: importación del padrón (.xlsx)

`POST /api/v1/tenant/units/bulk-import?organization_unit_id=…&modo=validar|confirmar`, con el archivo en el campo `archivo`. Archivo: `tenant/import.service.js`.

1	`multer`	Recibe el archivo en memoria (máximo 5 MB).
2	`leerFilas()`	`exceljs` abre el libro, lee los encabezados de la fila 1 y exige `codigo_interno` y `tipo_unidad`. Máximo 5000 filas.
3	`importarPadron()`	Valida **cada fila**: tipo de unidad reconocido, atributos con el tipo correcto (piso entero, área numérica, techada SI/NO), código repetido con otro tipo, código existente en el edificio con otro tipo, rol válido, documento, email y teléfono, el mismo documento con nombres distintos y filas duplicadas.
4	`advertencias`	Si un documento ya existe en el sistema con otro nombre, se avisa y se conserva el registro existente. No bloquea la importación.
5	`modo=validar`	Devuelve el resumen y la lista de errores por fila (fila, campo, mensaje) **sin guardar nada**.
6	`modo=confirmar`	Solo si no hay errores: en una transacción crea o actualiza las unidades, crea o reutiliza a los residentes por documento (`buscarOCrearPagador`) y crea los vínculos que falten. **Es todo o nada.**

16. Flujo: mora diaria y jobs programados

`jobs/index.js` programa tres tareas con `node-cron`, en hora de Lima y sin permitir que una ejecución se superponga con la siguiente.

Job	Cuándo	Qué hace
mora-diaria	00:01 todos los días	`recalcularMora()`: pasa a VENCIDO las cuotas pendientes cuya fecha ya pasó y recalcula la mora de todas las vencidas.
expiracion-pagos	Cada minuto	`expirarIntentos()` (capítulo 12).
notificaciones	Cada 15 segundos	`procesarNotificaciones()` (capítulo 17).

Cómo se calcula la mora — `calcularMoraCents()`

1	`SQL`	Toma las cuotas PENDIENTE o VENCIDO con vencimiento pasado y calcula los días vencidos (`CURRENT_DATE - fecha_vencimiento`, en hora de Lima). Las bloquea saltando las ocupadas.
2	`resolverConfig()`	Lee la regla del edificio: tipo, valor, días de gracia y tope.
3	`cálculo`	Días efectivos = días vencidos − días de gracia. **PENALIDAD_FIJA**: monto fijo si los días efectivos son más de 0. **INTERES_DIARIO**: monto base × % diario × días efectivos. Se aplica el tope si existe.
4	`UPDATE debts`	Solo si cambió algo. `monto_total` se recalcula solo en la base (base + mora − descuento).

EJEMPLO REAL DEL SEED

Cuota de S/ 300 vencida el 15 de agosto, revisada el 25 de octubre: 71 días vencidos − 3 de gracia = 68 días × 0,1 % × S/ 300 = **S/ 20,40** de mora. Como el cálculo parte de los días y no suma sobre lo anterior, correr el job dos veces el mismo día da el mismo resultado.

Las cuotas `EN_PROCESO` no se tocan mientras dura un intento de pago. Si el pago falla, al liberarlas se recalcula su mora con los días reales.

17. Flujo: notificaciones (outbox)

Ningún servicio envía mensajes directamente. Todos **escriben en una «bandeja de salida»** (tabla `notifications`) dentro de su propia transacción, y un job los envía después. Así, si un pago se revierte, su recibo tampoco existe; y si WhatsApp está caído, el mensaje se reintenta.

1	`encolarParaPagador()`	Busca el email y el teléfono del residente y crea una fila por cada canal de `NOTIFICACIONES_CANALES` para el que tenga contacto. Plantillas: AVISO_COBRANZA, RECIBO_PAGO, PAGO_RECHAZADO. Para administradores: DATO_INCORRECTO por email.
2	`procesarNotificaciones()`	Cada 15 s toma hasta 50 pendientes con `FOR UPDATE SKIP LOCKED` (si hubiera varios servidores, no se pisan).
3	`renderizar()`	Arma el texto según la plantilla. En AVISO_COBRANZA revisa que la cuota siga pagable y **crea el link de pago en ese momento**, para que el token nunca quede guardado en texto.
4	`providers.js`	`getProvider(canal).send()`. Hoy el proveedor de consola lo escribe en el log, con el destinatario parcialmente oculto.
5	`resultado`	Si se envía: ENVIADO. Si falla, reintenta con espera creciente (60 s, 120 s, 240 s…) hasta 5 intentos, y luego queda FALLIDO con el último error. Un aviso de una cuota ya pagada se omite.

18. Flujo: manejo de errores

En cualquier punto, los servicios **lanzan** un error y dejan de trabajar. Express lo lleva directo al `errorHandler` de `shared/errors.js`, que siempre responde con el mismo formato:

```
{ "error": { "code": "DEBT_IN_PROCESS", "message": "Esta cuota tiene un pago en proceso", "details": … }
}
```

Tipo de error	Respuesta	Ejemplo
Validación de Zod (`ZodError`)	400	`VALIDATION_ERROR` con la lista de campos y mensajes en español
Error de negocio (`AppError`)	Su propio código	401 `INVALID_CREDENTIALS`, 409 `DEBT_IN_PROCESS`, 402 `PAYMENT_DECLINED`
JSON mal formado	400	`INVALID_JSON`
Registro duplicado en PostgreSQL (23505)	409	`CONFLICT`
Cualquier otro	500	`INTERNAL_ERROR` con un mensaje genérico; el detalle va al log con el correlation-id

Si el error ocurre dentro de `withTransaction`, se hace **ROLLBACK**: la base queda exactamente como estaba. Por ejemplo, si al pagar tres cuotas la tercera ya estaba pagada, las otras dos tampoco quedan reservadas.

> **EN PALABRAS SIMPLES**
> Si algo falla a mitad de camino, el sistema deshace todo lo que había empezado, como si nunca hubiera ocurrido, y le explica al usuario qué pasó con un mensaje claro. Para el equipo técnico queda un número de seguimiento (correlation-id) que permite encontrar el detalle en el registro.

19. La base de datos y sus protecciones

Definida en `db/migrations/001_core.sql` (tablas) y `002_ledger_guards.sql` (triggers). Corre en PostgreSQL 16, en la base `tranky_db` dentro del contenedor Docker.

Relaciones principales



Tabla	Para qué sirve
`tenants` · `bank_accounts`	Administradoras, su configuración de comisión y las cuentas de las juntas
`organization_units`	Edificios, con sus reglas en `config_json`
`billing_units`	Unidades con `tipo_unidad` y atributos en `metadata_json`
`payers` · `unit_payer_relations`	Residentes (globales) y sus vínculos con rol; se desactivan, nunca se borran
`debts`	Cuotas; `monto_total` lo calcula la base (base + mora − descuento)
`transactions` · `transaction_debts`	Libro de pagos (ledger) y las cuotas que cubre cada pago
`payment_methods_vault`	Tarjetas tokenizadas (sin número ni CVV)
`webhook_events`	Eventos de la pasarela, únicos por proveedor e id
`pay_links` · `otp_challenges`	Links de pago y códigos OTP, guardados como hash
`notifications`	Bandeja de salida de mensajes
`settlements`	Liquidaciones a la junta (motor pendiente)
`data_issues` · `audit_logs`	Reportes de residentes y auditoría

Protecciones dentro de la propia base

- **Ledger inmutable:** triggers que impiden `DELETE` o `TRUNCATE` en `transactions`, `transaction_debts` y `settlements`, y cualquier cambio de montos o identidad de un pago. Un pago cerrado no puede «revivir»; un id de liquidación o de cargo no se sobrescribe.
- **Aislamiento entre administradoras:** claves foráneas compuestas `(id, tenant_id)`: una cuota no puede apuntar a una unidad de otra administradora, aunque el código tuviera un error.
- **Reglas de datos:** RUC de 11 dígitos, CCI de 20, periodos `YYYY-MM`, estados válidos, montos no negativos, unicidad de cuota por (unidad, periodo, concepto).
- **Rol sin privilegios:** la API se conecta como `tranky`, que no es superusuario; los triggers también la frenan a ella.

20. Estados de cuotas y pagos

Cuota (`debts.estado`)

PENDIENTE → **vence** → VENCIDO → **checkout** → EN_PROCESO → **webhook OK** → PAGADO

Transición	Disparador	Quién
(nueva) → PENDIENTE	Emisión de cuotas	`emitirCuotas()`
PENDIENTE → VENCIDO	Pasó la fecha de vencimiento	`recalcularMora()`
PENDIENTE / VENCIDO → EN_PROCESO	El residente inicia un pago	`checkout()`
EN_PROCESO → PAGADO	Webhook `charge.succeeded` con firma válida	`marcarExitoso()`
EN_PROCESO → PENDIENTE / VENCIDO	Rechazo, webhook `charge.failed` o expiración	`liberarCuotas()`

Pago (`transactions.estado`)

INICIADO → EXITOSO **o** FALLIDO · EXITOSO → **extorno** → REVERSADO

Hacia la app del residente se traducen así: INICIADO = «EN_PROCESO», EXITOSO = «PAGADO», FALLIDO = «FALLIDO», REVERSADO = «EXTORNADO». Un pago FALLIDO o REVERSADO no puede volver a otro estado; la base lo impide. Los extornos se registran como un pago nuevo de tipo `EXTORNO` que apunta al original.

21. Cómo extender el sistema

Agregar una pasarela real (por ejemplo, Culqi)

1. Crear `src/modules/gateways/CulqiGateway.js` que extienda `PaymentGateway` e implemente sus seis funciones.
2. Registrarla en `registry.js` con el nombre `culqi` y agregar su secreto de webhook en `env.js` y el `.env`.
3. Cambiar en el `.env`: `PASARELA_CARD=culqi`.
4. Configurar en Culqi el webhook hacia `/api/v1/webhooks/gateway/culqi`. El checkout y los estados no cambian.

Enviar WhatsApp o email de verdad

Crear un proveedor con la función `send({ canal, destinatario, mensaje })` y asignarlo en `notifications/providers.js`. Los reintentos y la bandeja de salida ya funcionan.

Agregar un rubro (por ejemplo, colegios)

Agregar una entrada en `config/rubros.js` con sus tipos de unidad (por ejemplo, ALUMNO), atributos y tipos de cuota. No hace falta tocar tablas.

Cambiar el esquema de la base

Crear `db/migrations/003_descripcion.sql` y ejecutar `npm run db:migrate`. Nunca editar una migración ya aplicada en producción.

Agregar un endpoint

1. Escribir la lógica en el `*.service.js` del módulo; si modifica varias tablas, usar `withTransaction`.
2. Agregar la ruta en el `*.routes.js`, con su esquema Zod y el tipo de token permitido.
3. Lanzar `AppError` para los casos de negocio; el `errorHandler` se encarga de la respuesta.
4. Agregar una prueba en `tests/api.test.js` y correr `npm test`.

22. Glosario

Término	Significado
API / endpoint	Servicio que responde a peticiones HTTP; cada endpoint es una dirección con una función, por ejemplo `GET /payer/units`.
Middleware	Función por la que pasa una petición antes de llegar a su destino (seguridad, registro, autenticación).
JWT / token	«Pase» firmado que la app envía en cada petición para demostrar quién es; vence a las 8 horas.
Tenant	Cliente B2B: una administradora con sus edificios. Sus datos están aislados de los demás.
Transacción de base de datos	Conjunto de cambios que se aplican todos juntos o ninguno (`BEGIN` / `COMMIT` / `ROLLBACK`).
`FOR UPDATE` (bloqueo)	Reserva filas mientras dura una transacción para que nadie más las modifique al mismo tiempo.
Idempotencia	Repetir la misma operación produce el mismo resultado una sola vez (no cobra dos veces).
Webhook	Aviso que la pasarela envía al backend cuando un pago se confirma o falla.
HMAC / firma	Sello criptográfico calculado con un secreto compartido; prueba que el aviso viene de la pasarela y no fue alterado.
Ledger	Libro contable de pagos: nunca se borra ni se modifica; los errores se corrigen con asientos inversos (extornos).
Token de tarjeta	Código que entrega la pasarela en lugar del número de tarjeta; el backend solo conoce ese código.
Outbox	Bandeja de salida: los mensajes se guardan primero y se envían después, con reintentos.
Job / cron	Tarea que corre sola en horarios definidos.
Hash	Huella irreversible de un dato: permite verificarlo sin guardarlo en claro (contraseñas, OTP, links).
Correlation-id	Número de seguimiento de cada petición para encontrarla en los registros.
Migración	Archivo SQL numerado que crea o modifica la estructura de la base de forma ordenada.

Documentos relacionados: `README.md` (instalación y comandos), `docs/API_FRONTEND.md` (contrato de la API para el frontend), `docs/Tranky_Resumen_Backend.pdf` (resumen ejecutivo) y `rules/FSD-002_Tranky_Portal_Residentes.md` (especificación funcional).